At the heart of OpenFGA is a schema, a declarative description of the application's authorization model. The schema defines types, which represent the core entities in your system (such as user, document, or folder), and for each type, it specifies relations and permissions.

- A *type* defines an entity and the kinds of relationships it can participate in. For example, a document might specify `owner`, `viewer`, and `editor` relations.
- A *relation* is a named link between two types, usually between a resource and a principal.
- A *permission* is a logical expression involving one or more relations, using set logic (such as `viewer` or `editor`, or `owner->manager`).

Once you've defined your schema, the relationship graph is stored as tuples. Each tuple encodes a single fact, like "Alice is a viewer of Document 123".

```
document:123#viewer@user:alice
```

These tuples form a directed graph of relationships among all entities. When a request comes in—such as asking, "Can Alice view Document 123?"—OpenFGA checks to see if the requested permission (like `can_view`) is valid, using the graph structure and the schema rules.

Unlike OPA or Cedar, OpenFGA doesn't evaluate conditional logic or analyze request attributes. It responds solely by walking the graph and applying relationship-based logic. This makes OpenFGA fast and predictable, especially in collaborative applications where relationships determine what actions are permitted.

8.3.2 Document access example

Recall this simplified access rule from ACME's Customer Collaboration system:

> *Permit access to a document if the requester is the viewer or the owner, or if they manage someone who owns it.*